System design.

→ It is a process of defining the architecture, components, modules, interfaces & data flow for a system to meet specific requirements.

→ It involves high level decisions about scalability, reliability, performance & maintainability.

⇒ Why is system design important ?

→ Scalability & reliability - ensures systems handle millions of users without failure.

→ Architectural thinking - involves CAP theorem, SQL vs NOSQL

→ Trade offs & decision making - Balances scalability, cost, speed & complexity.

→ Future proofing - prevents bottlenecks & allows smooth evolution of software.

Networking :—

IP address :- unique identifier assigned to devices on a network.

Primarily 2 versions → IPV4
→ IPV6.

Primarily 2 categories → Public
→ Private.

IPV4

→ 32 bit address format.
→ 4 sections, 8 bit address separated by '.'.

0 to 255 . 0 to 255 . 0 to 255 . 0 to 255.

IPV6

→ 128 bit address format.
→ designed for IOT, mobile networks, scalability.

Public IP address :-

→ assigned by internet service provider (ISP).
→ used to communicate over the internet.
→ unique world wide.

Private IPs :-

→ used within local networks.
→ cannot be accessed directly on the internet.

eg :- 10.0.0.0 – 10.255.255.255.

Private IP ⟶ NAT → Public IP → Global access.

NAT ↓

Network Address Translation

Role of IP in system design.

→ Scalability
→ Security
→ Load balancing
→ Cloud networking
→ Microservices.

DNS :-

- The phonebook of the internet.
- Translates human-readable domain names into IP addresses.

Types of DNS servers

- Root Name Servers - Top lvl servers handling .com, .org
- TLD Name servers - specific to domain like .com
- Authoritative Name servers - domain specific record.
- Recursive resolvers - handle queries on behalf of user, contacting multiple DNS servers if necessary.

User requests → Recursive resolver (ISP / Public DNS) → Root Name servers)

↓

Resolved IP Address returned ← Authoritative Name Servers ← TLD Name Servers

↓

Website Loads in browser.

DNS caching

→ reduces latency.

→ reduce load on DNS servers.

Where caching occurs:

→ browser cache. (stores previous lookups).

→ OS cache (Windows DNS cache).

→ Recursive resolver cache (ISP level).

TTL → time to live. determines how long a cached record is valid.

Step by step :-

→ user types a domain name.

→ browser cache check.

→ OS cache check.

→ Query to Local DNS resolver (ISP or configured DNS server)

→ If not found, Query goes to Root DNS servers.

→ TLD gives Authoritative DNS server details.

→ Authoritative DNS returns the IP address.

→ Response is cached & returned to the user.

Client - Server Model :-

→ Client request services, servers provide them.
→ enables ressource management.
→

Client :-

→ user - application that sends request.
→ Web browser / mob. app / API consumer.

Server :-

→ Process request & returns reponses.
→ Web server / db server / mail server.

Network :-

→ medium that facilitates communication b/w client & server
→ Internet / LAN / WiFi

Request - response cycle :-

HTTP example :-

→ browser request https://example.com
→ DNS resolves domain to IP address.
→ Browser sends an HTTP GET req. to web server
→ Webserver processes the request, fetches HTML, Queries db if needed.
→ sends an HTTP response with (200 OK) status code, & the webpage content.
→ browser renders the webpage.

Synchronous & Asynchronous Communication:

Synchronous :-

→ Client waits for a response before proceeding
→ used in REST APIs, traditional web apps.

Asynchronous App :-

→ Client doesn't wait for response & can perform other tasks.
→ web sockets for real-time chat.
AJAX for background request.

Stateless & Stateful servers :-

Stateless :-

- No memory of past interactions
- each request is independent.
- REST APIs, HTTP servers
- benefits :-
  - Scalability
  - easy caching
  - load balancing.

Stateful :-

- Maintain information across session
- Web sockets, Multiplayer games
- benefits :-
  - Personalization
  - seamless user experience.

Forward & Reverse Proxy :-

Proxy :-

- A proxy is an intermediary server b/w a client & another server.
- Proxies help with security, caching, traffic control, anonymity.

Forward proxy :-

→ Sits b/w client & Internet

→ client connects to fwd proxy instead of directly accessing websites.

→ Use case :-

- → Privacy & Anonymity.
- → Content filtering
- → Bypass Geo-restrictions
- → Caching -

→ Eg VPN, Web proxy.

Reverse Proxy :-

→ Sits b/w users & backend servers

→ Clients don't directly communicate with backend servers

→ Used for

- Load balancing
- Security & DDoS Protection
- SSL termination
- Caching & Compression.

→ eg → Cloud flare, AWS ALB, Nginx.

Load Balancing :-

Why Load Balancing ?

→ High availability – ensures system uptime even under high traffic.

→ traffic distribution – spreads requests evenly across servers

→ prevent Overload – avoid overloading a single server.

→ supports scalability

→ Improves performance

Types of Load Balancers :-

- based on Layer.
  - Layer 4 – operates at TCP/UDP level distributing request based on network level data. → suitable when speed & performance is priority.
    - decision made based on metadata.
  - Layer 7 – Operates at HTTP/HTTPS level, making routing decision based on request content.
    - inspect actual content
    - Ecommerce ~~sites~~ sites will use it.

- Based on deployment.

- Hardware load balancing - specialized devices.
- Software load balancing - Nginx, HAProxy, Envoy
- Cloud-based load Balancers - AWS Elastic cloud balancer
  GCP cloud balancing

Load balancing strategies :-

→ static load balancing → used when workloads are similar & predictable.

- distributes load based on pre defined rules / algos without considering current state of server.

⇒ round robin - distributes sequentially
⇒ least connections - directs to server with least load
⇒ IP hashing - based on client IP hashing.

- dynamic load balancing → monitors server conditions continuously.
  - least response time - sends to server with fastest response
  - Adaptive load balancing
    ↳ uses real time monitoring
  - Weighted load balancing
    → Assigns weights to servers based on capacity.

Choosing the right load balancer :-

- Layer 4 - when fast & efficient based on network level data.
- Layer 7 - need more intelligent routing based on headers or other content.
- Scalability needs - based on workload
- Security concerns - SSL termination & DDOS protection

API Gateway :-

- API - a bridge b/w client & server.
- API gateway is a centralised entry point for all API requests, handling authentication, routing, caching & security.
- key functions include
  - req. transformation
  - authentication
  - rate limiting
  - response handling
- benefits :-
  - security
  - rate limiting
  - Load balancing
  - Caching
  - Req. Transformation
  - Logging & monitoring

- provides Authentication — Supports OAuth, JWT, API key
- DDOS Protection - blocks excessive requests from mal. software.
- BOT Protection - uses rate limiting & CAPTCHA to filter out bot traffic.
- TLS termination - Handles SSL encryption to secure communication.
- Rate limiting - limits the no of requests per user per sec.
- Throttling - controls traffic during peak loads to avoid system crashes.
- helps prevent abuse, protect backend services & ensures fair usage.
- caching → in memory caching
  - → response caching
  - → edge caching

API Composition & aggregation

- Combines multiple API calls into a single request.
- Useful in microservices architecture where one client req, may need data from multiple services.

Popular API gateways

- Open source - Kong, NGINX, Traefik.
- Cloud
  - AWS API Gateway.
  - Google APIgee.
  - Azure API management.

When to use API gateway?

- best suited for microservices architecture.
- APIs requiring security, rate limiting & monitoring.

CDN - Content Delivery Network :-

- a globally distributed network of servers that work together to deliver content efficiently

- CDNs exists to
  - reduce latency
  - enhances content availability
  - improves load handling
  - security.

Problems without CDNs,

- high latency due to geographic distance.
- Overloaded origin servers
- bandwidth constraints & slow load times.

Components of a CDN :-

- Origin servers :- store the original content.
- edge servers (Points of Presence POPs) - store cached content closer to users
- request routing system - directs users to nearest POP.

How CDN works?

- User request content (website / video / API / data).
- CDN directs the request to nearest edge server based on geographic loc / latency / server load.
- Cache hit - if the content is available on the edge server, it is delivered instantly.
- Cache miss - if the content is not cached, the request is routed to origin server, fetched & stored at edge server for future use.

Feature of CDN.

- Compression & Optimisation
  - Techniques like Gzip, Brotli, image optimisation reduce file size for faster delivery.
- Security features
  - DDoS Protection, SSL/TLS

Cache invalidation techniques.

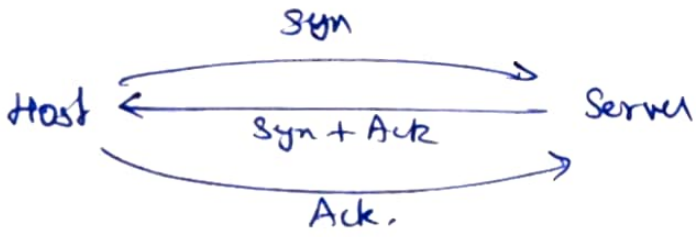
- manual purge
- Stale - while revalidate.
- cache control headers.

Protocols

TCP & UDP :-

TCP:-

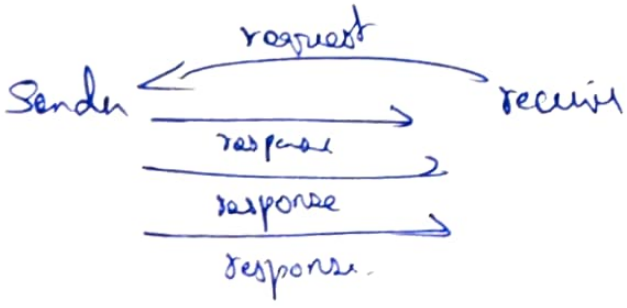
- Connection oriented protocol, a connection should be established before data transfer, which is achieved by 3 way handshake.



- ensures reliable, ordered, error-checked communication.

UDP:-

- connectionless protocol.
- faster, but no delivery guarantee
- no retransmission of lost packets.



	TCP	UDP
reliability	high	low
Speed	low	high
Connection type	Connection-oriented	Conectionless
ordering	Ordered	cannot ensure order
error handling	error handling + retransmission is done.	no error handling mechanism
Overhead	high	low
use case	Web browsing, file transfer, email, Db communication	Video streaming, Online gaming, DNS lookups

HTTP - hypertext transfer protocol.

- works over TCP/IP
- Port 80 - HTTP
- Port 443 - HTTPS.
- text. - based protocol.
- stateless - each request is independant of another.
- supports multiple methods (GET, POST, etc)

HTTP - Working

- Client - server model.
  - Client makes an HTTP request
  - Server processes the request & sends back a response.

- Components of an HTTP request.
  - Method - defines the action (GET / POST / etc).
  - URL - The resource being requested.
  - Headers - Metadata.
  - Body - data sent in POST / PUT requests.

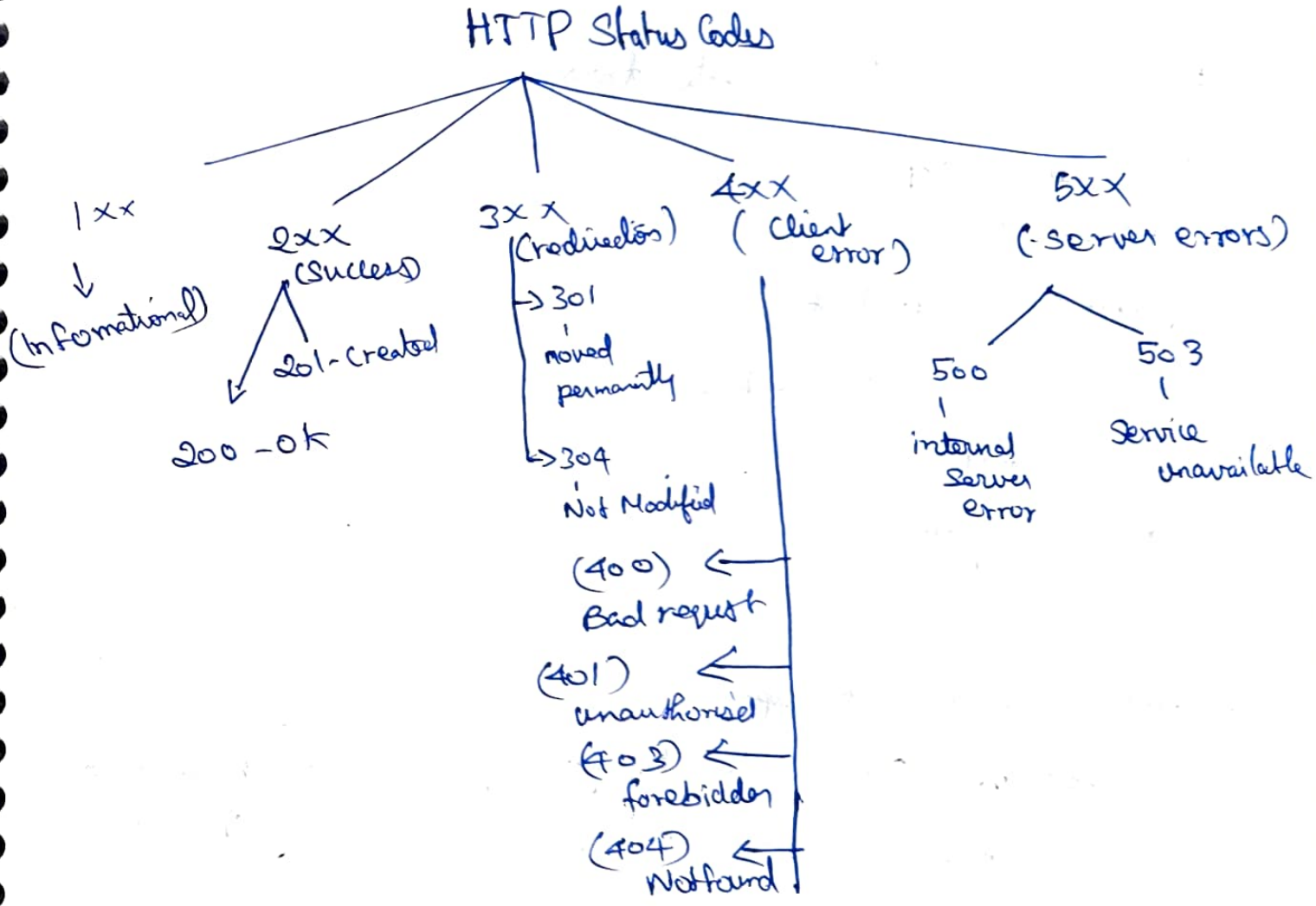
- Components of an HTTP response
  - Status code - (indicates failure / success)
  - Headers - (metadata)
  - Body - (actual response).

Request - response cycle :-

1. Browser sends a request.
2. Server processes the request.
3. Server generates a response & sends it back.
4. Browser renders the response.

Since HTTP is stateless, we cannot maintain user sessions. So we have to take measures to handle State.

- Cookies - Small pieces of info stored in browser.
- Sessions - Server side storage of user state
- Tokens - Used for authentication & authorization.



HTTPS -

- using SSL / TLS encryption.
- used for secure websites, online banking and e-commerce
- works on port 443

REST & Restfulness :-

REST - representational state transfer.

- based on http
- stateless
- platform independant
- client - server architecture
- cacheability
- layered system.

Restful API design principles.

- GET /user/Eid3 → to fetch specific user } resource based approach.
- GET /users → to fetch all users.

versioning can be implemented, making it backward compatible

eg: /v1/users.

JSON
- light weight
- faster parsing
- readable

XML
- legacy system
- data validation needs (schemas).

Real-time Communication Protocols :-

↳ continuous exchange of data with minimal latency.

approaches :-

- polling ⇒ client constantly interacts with server
- web sockets ⇒ a persistant connection with server.
- Server sent Events ⇒ Unidirectional streaming from server.
- long Polling ⇒ server holds request until new data is available.

Web sockets :-

→ a persistant, full-duplex connection btw the client & server over a single TCP connection.

→ steps

- Client requests an upgrade to web sockets.
- Server accepts & keeps the connection open.
- data is exchanged in real-time using frames.
- either party can close the connection when done.

Use cases of Websockets :-

- Live chat Applications
- Stock Market price updates.
- Multiplayer Online games.
- Collab tools.

Long-polling :-

- Client sends a request to the server & waits until the server has new data to respond with.

working :-

- Client makes a HTTP request.
- Server holds the request until data is available.
- Server responds with new data.
- Client sends another request

use cases :-

- Slack chat.
- IOT devices.
- notifications in Twitter.

gRPC -> A - high performance, binary protocol optimised for microservices & real-time communication

GraphQL -> A flexible query language that allows clients to fetch only the data they need.
(frontend optimised)

GrPC - working

- built on HTTP2.
  - multiplexed requests - multiple calls over 1 connection
  - Compression
  - full duplex streaming
- uses protocol buffers for fast serialization

Software Architecture

- high level structure of a software system.
- The architecture we choose has impact on.
  - scalability
  - maintainability
  - performance
  - system behaviour.

Monolithic architecture :-

- a single unified system where all components are tightly coupled & work as a single unit.
- easier to manage in smaller application.
- hard to maintain as code base grows.
- a single bug can takedown the entire system.

Layered (N-tier) Architecture :-

- a system is split into multiple layers.
- clear separation of concerns
- easier to scale & maintain
- might result in tight coupling between some of the layers.
- commonly used in enterprise applications, banking etc.

Microservices Architecture :

- a system built as a collection of small independent services, each focused on a separate business capability.
- independent services can be scaled, deployed & developed separately.
- better fault tolerance.
- Need robust devops & automation pipelines.
- used in large scale applications, e-commerce platforms, Cloud-based systems.

Event-driven architecture :-

- A system where components communicate through event msgs instead of direct calls, enabling loose coupling.

- excellent for handling asynchronous workflows
- difficult to ensure data consistency across services.
- used in IOT apps, real-time systems, financial trading apps

Multitier Architecture :-

- a design pattern that structures apps into multiple layers, each responsible for specific functions.

2tier :-

- client & db layer.

↙ ↓

UI & app logic — stores & retrives data

Pros :-

- simple to implement.
- faster for small-scale apps.

Cons :-

- poor scalability
- security risks.

3tier :-

- UI, Business layer, db.

UI ⇄ Business layer ⇄ db.

- improves scalability & security
- easier maintanence

N-tier:-

- extends beyond 3-tier by adding additional layers, like- caching, gateway, microservices etc.
- handles high traffic & complex business logic.
- allows independant scaling of different services.

How tiers impact system performce & scalability :-

- Latency conciderations :-
  - more layers → higher latency.
  - handled by - caching, Load balancing

- Scaling strategies :-
  - Vertical scaling - adding resources to a single server.
  - horizontal scaling - adding more servers, distributing load.

Microservices Architecture :-

- a software design pattern where apps are structured as a collection of small, loosely coupled services, each responsible for a specific fn.

How to Identify m.s ?

- Business capability
- Single responsibility
- Data Ownership
- Independantly deployable

How to structure Microservices ?

- decompose by business domain use Domain driven design to group services logically
- define clear apis
- choose right granularity, avoid making microservices too large or too small.
- use logging monitoring & tracing to track service notations & interactions.

Communication in micro services

=> Synchronous

- REST APIs
- gRPC

=> Asynchronous

- Event driven msging
  - Kafka, RabbitMQ

Event driven Architecture :-

- A system where components communicate through events instead of msgs.
- Asynchronous - unlike waiting for response, can continue working
- Loose coupling
- Scalable.
- enhances system responsiveness
- enables real time event processing

Pub-sub :-

- events are broadcasted to multiple subscribers.
- each subscriber gets the event once.
- RabbitMQ, AWS SNS.
- Great for real time notification.

Event Streaming :-

- stored & consumed in order.
- consumers process events at diff. times.
- Kafka., AWS Kinesis.

Best practices :-

- use idempotent event processing to avoid duplicates
- implement dead letter queues for failed msg.
- choose the right event broker based on system needs
- ensure event versioning to handle schema changes.

Web Concepts in system design.

Web sessions :-

- HTTP is stateless, meaning each request is independant
- It doesn't remember past interactions.

Techniques for maintaining state :-

- session based authentication
  - server maintains session state
  - client holds only a session ID
- Token based authentication (JWTs, OAuth Tokens)
  - session state is embedded within token itself
  - server doesn't need to track user sessions.

Session based Authentication :-

- server side session storage
- Cookies for session ids.
- User logs in → server creates a session & assigns sessionID
- session data is stored in server side, session id is sent to client.
- client stores session details in cookie.

Token-based Authentication :-

- encodes session data in a self-contained token
- no need of server side storage.
- used in mordern Stateless Authentication.

Data exchange & storage formats :-

Serialization - Converts complex objs into a format that can be easily transferred.

deserialization - converting it back into an object.

essential in distributed systems & inter process communica

JSON -> human readable, lightweight, key-value structure

XML -> Structured but verbose, used in legacy systems.

Protobuf -> compact, used in gRPC, binary format developed by google, needs schema definitions.

serialisation in APIs :-

REST - Json

GRPC - protobuf.

XML - SOAP.

Serialisation in caching & data storage :-

Redis & Mem cached - store serialized Json / Protobuf.

Databases - NoSQL databases like Mongo db

Big Data - Protobuf for efficient Storge & schema evolution.

readability vs Efficiency vs Compactibility.

readability → Json & XML are human readable but inefficient.

Efficiency → Protobuf & Avro are compact, reducing bandwidth usage.

Compactibility → XML supports schema evolution, Json has limited support.

performance conciderations :-

- Serialization choice impacts bandwidth, CPU & memory usage.
- JSON / XML → Larger payloads & slower parsing.
- Protobuf → Smaller payloads, faster parsing needs schema.

CORS - Cross-Origin Resource sharing & Web security -

The problem - browsers enforce Same-origin Policy
blocking cross-origin requests by default.

CORS is a mechanism that allows secured cross-origin communication.

CORS is server driven. The server must explicitly allow access.

Two types of requests :-

- Simple requests - GET, POST (Without Custom headers)
- Preflight requests - NEEDED for PUT, DELETE or custom headers.

CORS headers control.

- Access control Allow Origin (origins can access)
- Access Control Allow methods (HTTP methods allowed)
- Access Control Allow Headers (custom headers that can be sent).

Preflight requests & CORS headers.

- some requests need pre flight checks. (sent via options requests).
- browser first sends a pre flight request before making actual request.
- If the browser responds with valid CORS headers, the browser allows actual request.

Handling CORS in APIs.

- CORS in REST
  - JSON based apis commonly use CORS
  - Configure headers properly in backend frameworks

Alternative to CORS & role of API gateways.

- Alternatives
  - Reverse Proxy
  - API gateways handling cors.

Scalability :-

Introduction to scalability :-

Scalability - ability of system to handle increasing amount of work or its potential to accomodate growth.

Types of Scaling :-

Vertical scaling - Add more CPU / RAM to one server.

Horizontal Scaling - Add more servers to distribute the load.

Scaling Strategies :-

diagonal scaling - start vertical, then go horizontal

Vertical - upgrade one machine
- simple to implement
- risk of single point of failure.

Horizontal - add more machines
- requires load balancer, stateless design.
- complex setup.

diagonal - start vertical, go horizontal.
- common in cloud-native apps.
- cost effective + long term ready.

Load balancer :-

Why? ⇒
- high availability
- traffic distribution
- prevents Overload.
- improves performance.
- handles failures
- supports scalability.

Types :-

i) Based on layer
- Layer 4 ⇒ Operates on TCP/UDP level, distributing requests based on network-level data.
- ideal when speed + performance is priority.

- Layer 7 (App layer).
  - operates on HTTP/HTTPS level, making routing decisions based on request content.

ii) Based on deployment

- Hardware load balancers
  - Specialized devices
- Software load balancers
- Cloud Load balancers.

Load balancing strategies :-

Static load balancing - used when workloads are predictable.
- round robin
- least connections
- IP hashing

Dynamic load balancing - takes realtime server conditions into account.
- least response time
- Adaptive load balancing
- weighted load balancing

Auto Scaling in cloud environments :-

Autoscaling - automatic adjustment of compute resources based on load

how autoscaling works ?

- triggers
  - CPU usage
  - Memory
  - req / rate
  - Queue length

Scaling policies → Reactive - based on thresholds
→ Predictive - based on trends

monitoring & proactive scaling :-

- use metrics like
  - CPU, memory, network
  - Queue depth, Custom KPIs

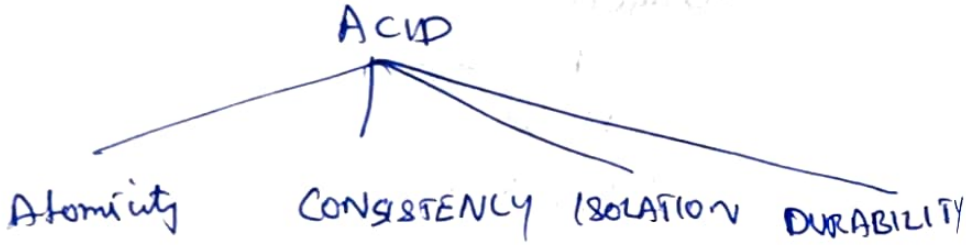
Database and Storage.

Structured data - rows/columns, predefined schema.

Unstructured data - No schema, flexible format.

Storage properties :-

- durability
- availability
- consistency
- Atomicity



Factors to consider designing storage.

- scalability
- reliability
- performance

CAP Theorem :-

- Consistency - every read gets the latest write
- Availability - every req receives a response
- Partition Tolerance - system continues despite network failures

- No system can have all 3 at the same time.

Since network partitions are unavoidable, especially at scale, real-world systems must choose between consistency & availability during a partition.

Types of Systems based on CAP trade offs :-

- CP (Consistency + Partitioning)
  - prioritizes data correctness over availability.
  - during a partition, the system may reject requests to avoid inconsistent reads.
  - Not always available, but when it is - data is guaranteed to be correct
  - example - Hbase - strongly consistent. If a node can't confirm a write across replicas, it won't serve even if it means being unavailable briefly.
  - When to use ⇒ financial systems, banking apps, anywhere where data integrity is critical.

- AP (Availability + Partition tolerance)
  - prioritises system uptime over consistency
  - during a partition, the system will serve requests, even if they return stale or eventually consistent data
  - eg - dynamodb - inspired by amazon's dynamo model, which uses consistency by default for high availability.
  - when to use: social media feeds, product catalogs, content delivery - where being up is more important than perfect accuracy.

- CA (consistency + availability) - The "Unicorn".
  - only possible if no network partition ever occur.
  - in practice, not achievable in distributed systems that need to ~~tool~~ tolerate network faults.
  - eg - rel. db in stand alone mode could be considered

Core concepts of relational db :-

- schemas - predefined structures of data.
- Joins - combine data across multiple tables.
- ACID Properties :
  - Atomicity - all or nothing
  - Consistency - data integrity maintained
  - Isolation - transactions dont interfere
  - Durability - changes survive system failures.

Rel. db not ideal for.

- rapidly changing or schemaless data.
- Large scale horizontal scaling
- flexible or nested data.

NoSQL → schemaless or dynamic schema.

Types:
- document - JSON like structure
- Key value - key based lookups
- columnar - Store data by column, optimised for analytical queries
- Graph - Store entity & relationship as nodes and edges.

BASE properties in NoSql.

Ba - basically available - always return a response

S - soft state - system state may change overtime.

E - eventually consistent - data will be consistent, eventually

CAP Theorem:

SQL tends towards CP. Consistency & partitioning

NoSQL tends often choose AP

Scaling strategies - SQL vs No SQL.

Vertical scaling - traditional SQL db.

- add more CPU / RAM / SSD to single db server.
- common with rel. db.

horizontal scaling - Modern NoSQL db

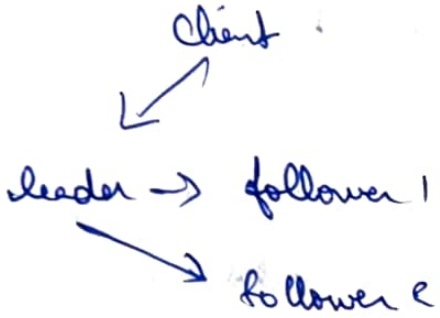
- add more db nodes to distribute data & load.
- common with No SQL db

Replication :-

- copying data from node to another for redundancy & performance.
- benefits :
  - fault tolerance
  - data availability
  - read performance improvement.
- CAP theorem
  - replication may favour availability.

Leader - follower replication :

- Writes go to leader
- reads from followers



- Async replication may cause a possible lag.

Sharding :-

- splitting data across multiple db to scale horizontally

Horizontal - rows are distributed across shards

Vertical - tables or columns are split across shards based on function or access pattern

Sharding strategies :-

- range based sharding
- hash based sharding
- Geo based sharding

Object storage in modern systems :-

Object storage is a storage architecture that manages data as ~~objects~~ objects not file or blocks.

Each object contains :

- data initself
- a unique identifier
- meta data.

- suitable for distributed & unstructured data.

bucket - a logical container for storing objects.

Common use cases :-

- media storage
- backups & archives
- data lakes for analytics
- static website hosting
- IoT & ML data pipelines.

Performance Considerations :-

Latency - object storage has higher latency than block/file

Throughput - designed for massive parallel access.

Consistency - eventual consistent

Access pattern - Suited for write once, read - many workload

Cost Considerations :-

- storage class tiers : standard, Infrequent Access, Archive
- Charges : storage, requests (PUT/GET), egress bandwidth

File systems :-

- defines how data is stored & retrived on disk.
- handles file metadata, directories, permissions.

Characteristics :

- hierarchical structure
- limited scalability
- suited for single node system

distributed file system :-

- allows file access across multiple nodes
- ~~allows~~ appears as a single file to user.
- scalable & reliable.

ex- hadoop

System performance

performance - efficiently a system meets its functional requirements under load.

key dimensions

- Speed
- capacity
- efficiency

Latency - time taken to process 1 request.
- affects responsiveness

Throughput - No of requests processed per second
- Affects scalability.

Measuring performance :-

SLA - Service Level Agreement

External contractual guarantee.

SLO - Service Level Objective

Internal target.

SLI - Service Level Indicator.

Actual metric Value.

Performance testing

- Load testing - Normal load conditions
- Stress testing - Beyond normal limits
- Spike testing - Sudden large loads
- Endurance testing - Over extended time.

Performance monitoring. - done continuously

- Application performance monitoring tools.
- Logs & metric tools.
- track
  - latency
  - throughput
  - error rates.

Caching

- reduces latency
- critical in low latency, high throughput architectures.

Types :-

-> Client side - browser memory
-> Server side - App level memory
-> CDN caching - static content cached close to users
-> database caching - result set caching, materialized views

Caching strategies

- Write through ⇒ db & cache are updated at same time.
- Write behind ⇒ Write to cache, db updated asynchronously
- Lazy loading ⇒ Cache is populated only on demand.
- Manual caching -> dev decides when to cache or evict.

Cache eviction policies

- LRU - least recently used.
- LFU - least frequently used
- FIFO - remove oldest item added
- TTL - Automatically expire items after a fixed time.

Redis - in memory key value store.
- supports TTLs.
- used for caching, queues, sessions
- Opensource & widely adopted

Asynchronous messaging :-

- Loose Coupling - decouple producers from consumers
- improved performance - producers dont block waiting for consumers
- Scalability - consumers can scale independently

Messaging systems :-

Message - data packet

Producer - sends msg

Consumer - receives & processes msg.

Queue/broker - stores & delivers msg

Topic - Logical channel for msg delivery

Ack - Acknowledgement.

When to use Queues in Architecture ?

→ workloads are bursty
→ decoupling b/w services
→ buffer spikes in traffic
→ for background jobs
→ for rate limited/expensive operations

Common use case of msging queues.

- Order processing
- Logging & monitoring
- rate limiting
- Email/sms notification
- ETL pipelines.

Best practices :-

- use idempotent consumers
- implement dead letter queues (DLQs)
- Monitor queue length & processing time.
- handle retries & failures gracefully
- Choose delivery semantics based on need
- secure your message brokers (auth, encryption)

Concurrency & parallelism :-

Concurrency -> when multiple start, run & complete in overlapping time periods

- its about managing multiple tasks efficiently, esp. on a single CPU core.
- its all about task management

Parallelism -> when multiple tasks are executed simultaneously on multiple CPU cores

- it is about actual simultaneous execution to speed up performance.
- needs multi-core for true parallelism.

Process -> have their own memory space.
-> heavier to create & switch
-> isolated

Thread -> shared memory within a process
-> lightweight & faster

Thread pool → pre created threads reused for multiple tasks.

Worker models :

- Tasks are distributed to idle workers from a shared queue
- improves scalability & CPU utilization.

Async processing :-

- Avoid blocking threads on I/O
- improves throughput.

techniques :-

- Async / Await
- Promises / futures
- Message queues

Race condition :- - multiple threads access & modify shared data concurrently
- causes unexpected behaviour or corruption

deadlocks - Two or more threads wait for each other to release resources

Index → data structures that improve query performance by reducing the amount of data scanned.

Types of Indexes :-

B tree indexes - Common for exact match & range queries

Hash Indexes - best for equality comparisons

Full Text Indexes - used for searching large textual data.

Bitmap Indexes - Suitable for low cardinality columns

System reliability :-

- ability of a system to operate continuously without failure
- includes
  - correctness
  - consistency
  - fault tolerance

MTBF - Mean time between failures - Avg time a system works before failure

MTTR - Mean time to recovery - Avg. time to recover from failure.

High MTBF + Low MTTR → High reliability

SLA - Contractual guarantee about system performance

reliability in distributed systems.

- Challenges
  - network partitions
  - node failure
  - eventual consistency
- Solutions
  - Use CAP-aware design
  - Ensure fault isolation
  - implement replication & concensus algorithms.

redundancy is essential to ensure the system doesn't suffer single point of failure.

Types of redundancy

- hardware
- Network
- Services

N+1 redundancy - one extra instance, more than what is necessary.

Active - Active redundancy - Multiple nodes work together, each handling requests.

Active - Passive - One active node others on standby. activated on failure.

Graceful degradation :-

- System still operates at a reduced capacity during failures.
- during high traffic, disable non-essential features.

Self healing systems - automatically repair or replace failed components.
- Commonly used in cloud environments for fault tolerance.

Backup & recovery :-

Types of backup

- Full backup – copies all data
- Incremental backup - backs up only changes since last backup
- differential backup – back up changes since last full back up.

Recovery types :-

- Cold - backup stored offline, high downtime
- Warm - some resources pre-provisioned, moderate downtime
- hot - fully redundant, minimal downtime.

RTO - recovery time objective
- how quickly can the system recover.

RPO - recovery Point Objective
- how much data loss is acceptable.

Quorom based design - minimum no of servers that must acknowledge a distributed operation to be successful before its marked a success.

Security in System design :-

- in distributed systems, more entry points ⇒ more Vulnerabilities

→ security consideration :-

- data in transit & storage.
- Authentication & Authorization
- Secure APIs & endpoints
- Node & network level protection.

CIA triad :-

C → Confidentiality - prevent unauthorised access.

I → Integerity - prevent data tampering

A → Availability - Ensure system uptime & access

Threat modeling ⇒ define potential attackers & what they want.

to consider ⇒ - Attack surface
- entry points
- Assets to protect.

STRIDE model :-

Spoofing - act of impersonating another user to gain unauthorized access.

Tampering - Unauthorized alteration of data.

Repudiation - The ability to deny action taken, complicating accountability.

Information Disclosure - unauthorized access to confidential info

Denial of service - disrupting access to services or resources.

Common attacks :-

DDOS → distributed denial of service
- flooding system with traffic to disrupt service
- targets : availability
- Mitigation
  - rate limiting
  - traffic scrubbing.
  - Autoscaling & failover strategies.

Man In the middle (MITM) attack

- attacker interupts communication
- targets confidentiality & integerity
- Protection :
  - HTTPS
  - Certificate pinning
  - VPNs

Spoofing Attacks

- impersonation of another user
- types : IP spoofing / email spoofing / DNS spoofing
- defense :
  - multifactor authentication
  - token based auth
  - IP whitelisting

Injection Attacks

- attacker sends malicious input to execute unwanted commands.

- Impacts - data integerity & Confidentiality
- Mitigation
  - Input validation
  - parameterized queries
  - WAF (Web Application Firewall).

Security in SDLC :-

Phases :

- Requirements - threat modelling
- design - secure architecture
- development - secure coding
- testing - security tests, fuzzing
- deployment - secrets management
- Maintanence - patch management

Authentication & Authorisation :-

Authentication : verifies who the user is

Authorisation : granting the user permission to access specific resources based on their identity.

Common authentication methods

- basic authentication - Username + password,
- OAuth 2 - delegated access protocol, allowing third party services to access user data without exposing credentials
- Open ID connect - An identity layer built on top of OAuth 2 for authentication, often used for Single sign on.
- JWT - Token based authentication, used in stateless apps & APIs.

Access Control Models.

RBAC -> - Role Based Access Control
- assigns permissions based on user roles.
- simpler to manage but less flexible.

ABAC -> - Attribute Based Access Control
- grants access based on attributes
- more fine grained but more complex.

DAC -> Discretionary Access Control.
- Owner of a resource defines access controls

MAC -> Mandatory Access Control
access control decisions are made by a central authority based on security policies.

SSO & Identity federation.

SSO - A user authentication process that allows a user to access multiple apps. with a single set of credentials

Identity federation - A system where multiple identity providers can be used to authenticate users across different platforms.

Encryption → ciphering the data

2 types

- Symmetric - one key ; fast & used for large data
- asymetric - public & private key

TLS/SSL & https. :-

→ https ⇒ http over tls

→ tls handshake → key exchange + cipher negotiation.

hashing & salting passwords :-

→ hashing ⇒ one way transformation

→ Salting ⇒ adding random data to prevent rainbow table attacks

→ add random text before hashing & the hash & store the hashed data.

Firewall → filters traffic based on IP, port + protocol.

types – network based, host based, cloud firewalls –

Reverse proxies – route, mask backend identity

Firewalls - filters traffic at perimeter

reverse proxies – route traffic securely

Ratelimiting → per user, per - IP request caps.

throttling → graceful degradation under load.

IP filtering → allow/block set of IPs

Zero trust Security model :-

- never trust always verify -
- Auth every request
- Microservices – mutual TLS

Redis

- Single threaded, in-memory data structure server.
- Single threaded, while one operation executes, others wait.
- key value store
- 3 main peculiarities:-
  - holds its database entirely in memory, using disk only for persistence.
  - rich set of data types.
  - replicates data to any no of slaves.
- all redis operations are atomic, the clients if operates concurrently, redis server receives the updated value.
- it is a multi-utility tool & can be used in a no of use cases such as caching, messaging queues, natively supports Publish/Subscribe, any short lived data in your application such as web application sessions, hit counts etc.

- Redis values can contain more complex data structures with atomic operations defined on those data types
- in memory database but persistent on disk database
- very high write & read speed is achieved with the limitation of datasets that can't be larger than the memory.